Kuku AI Generalist Assignment — Problem 1: Find the Promos

Prototype: Kuku Promo Finder | **Input:** 50 screenplay episodes supplied in the assignment pack

1. Problem framing and scope

Editors currently watch every episode and manually identify 20–60 second promotional moments. I built a repeatable first-pass tool that scans timestamped screenplay beats and ranks candidate windows for human review. It returns episode number, start/end timestamps, an opening-beat cue, an ending-beat cue and a short reason. It does not edit video or claim to predict actual ad performance.

2. Assumptions

• The supplied screenplay timestamps are episode-relative and use MM:SS.xx-style time notation.
• A promo can be a continuous segment; this prototype focuses on continuous windows to keep the first version reliable and easy to inspect.
• The editor makes the final decision. A strong candidate must still make sense to a first-time viewer and have a genuine unresolved ending.

3. How it works

The Python parser extracts timed dialogue/action/camera beats. It scans overlapping windows of 25–60 seconds in five-second increments. A transparent heuristic score combines (a) hook-related signals in the opening three seconds, (b) emotional/dramatic language throughout the window, (c) unresolved-ending signals in the final eight seconds, and (d) the density of timed beats. It keeps up to three non-near-duplicate candidates per episode and ranks the combined list. The Streamlit app accepts the screenplay ZIP and exports JSON and CSV.

4. Initial run and quality checks

The supplied pack contains 50 episode text files. The current run produced 148 candidate windows (up to three per episode). The highest-scoring initial candidate is Episode 3, 00:35–01:30 (score 33.8). The raw JSON/CSV and a top-candidate run summary are included with the prototype.

5. Where it picks badly / limitations

This is a baseline, not a semantic story editor. Keyword matching can mistake generic words such as “why”, “never” or “secret” for a compelling hook/cliffhanger; it can miss visual-only reveals, sarcasm, emotional subtext and context that depends on earlier episodes. A high beat count can also reward dense but confusing scenes. The tool does not yet test whether a segment is understandable without context, join moments across episodes, or use viewer-performance data. These are explicit human-review checks, not solved claims.

6. Next iteration and scale

For production, retain the timestamp parser and replace the heuristic scorer with a swappable LLM evaluator that receives each candidate plus brief preceding/following context. Require structured scores for hook strength, standalone clarity, emotional escalation and cliffhanger strength, with quoted evidence and confidence. Editors should approve/reject candidates; those labels become evaluation data. Batch processing, caching and parallel episode analysis make weekly show ingestion practical. Track editor acceptance rate, time saved per episode and promo watch-through/retention after publishing.

7. Tools and honest AI use

Python and Streamlit implement the parser, scoring and interface; ChatGPT assisted with code scaffolding and review. The scoring is deterministic and does not call an external model/API. One known AI-assisted implementation weakness is that keyword matching can label a line as a cliffhanger without understanding its narrative context. The model/scorer can be swapped later without changing the upload/export workflow.